Git Branching Complete Guide

Step 1: Check Git Installation

git --version

Example:

git version 2.48.1

Step 2: Open Your Project

Navigate to your project folder.

cd /path/to/project

Example:

cd C:\Users\Hari\Desktop\MyProject

or

cd /home/ubuntu/project

Step 3: Check Current Branch

git branch

Output:

* main

Step 4: View All Branches

Local branches

git branch

Remote branches

git branch -r

Both local and remote

git branch -a

Example

```
* main
  dev
```

Remotes

origin/main
origin/dev
origin/testing

Step 5: Get Latest Changes from GitHub

Before creating a new branch, update your local repository.

git pull origin main

If your default branch is master:

git pull origin master

Step 6: Create a New Branch

Syntax

git branch branch-name

Example

git branch feature-login

Verify

git branch

Output

```
* main
  feature-login
```

Step 7: Create and Switch to New Branch (Recommended)

Instead of two commands, use one:

git checkout -b feature-login

or the newer Git command:

git switch -c feature-login

Output

Switched to a new branch 'feature-login'

Step 8: Switch Between Branches

Using switch

git switch main

or

git switch feature-login

Older method

git checkout main

Step 9: Verify Current Branch

git branch

Example

main
* feature-login

Step 10: Make Changes

Edit your files.

Example:

index.html
app.js

style.css

Step 11: Check Modified Files

git status

Output

modified: app.js
modified: style.css

Step 12: Add Files

Add all files

git add .

Add one file

git add app.js

Step 13: Commit Changes

git commit -m "Added login feature"

Step 14: Push New Branch to GitHub

git push origin feature-login

First time only

git push --set-upstream origin feature-login

or

git push -u origin feature-login

Step 15: Pull Latest Changes

Current branch

git pull

Specific branch

git pull origin main

or

git pull origin feature-login

Step 16: Fetch Remote Changes Only

git fetch

This downloads updates but does **not** merge them.

Step 17: Switch Back to Main

git switch main

Update it

git pull origin main

Step 18: Merge Feature Branch into Main

git merge feature-login

Push merged changes

git push origin main

Step 19: Delete Branch

Delete local branch

git branch -d feature-login

Force delete

git branch -D feature-login

Delete remote branch

git push origin --delete feature-login

Step 20: Clone Repository (If You Don't Have the Code)

git clone https://github.com/username/project.git

Go into the project

cd project

Step 21: See Remote Repository

git remote -v

Output

origin https://github.com/user/project.git (fetch)
origin https://github.com/user/project.git (push)

Daily Workflow (Most Common)

```
# Go to project
cd project

# Update main
git switch main
git pull origin main

# Create feature branch
git switch -c feature-login

# Make changes
git status

# Stage changes
git add .

# Commit
git commit -m "Added login page"

# Push branch
git push -u origin feature-login
```

Useful Git Commands

Task	Command
Check status	`git status`
Current branch	`git branch`
All branches	`git branch -a`
Create branch	`git branch branch-name`
Create & switch	`git switch -c branch-name`
Switch branch	`git switch branch-name`
Pull latest code	`git pull origin main`
Fetch only	`git fetch`
Add files	`git add .`
Commit	`git commit -m "message"`
Push	`git push origin branch-name`
Delete local branch	`git branch -d branch-name`
Delete remote branch	`git push origin --delete branch-name`

Real-world Team Workflow

Suppose the repository has a `main` branch and you need to work on a new feature:

Update your local `main`:
git switch main
git pull origin main

1.

Create a feature branch:
git switch -c feature-user-profile

2.
3. Make your code changes.

Commit your changes:
git add .
git commit -m "Add user profile page"

4.

Push the branch:
git push -u origin feature-user-profile

5.
6. Create a Pull Request on GitHub.

After the Pull Request is merged, update your local `main`:
git switch main
git pull origin main

7.

Delete the feature branch if it's no longer needed:
git branch -d feature-user-profile

8.

This is the standard workflow followed by most software development teams.